

Лабораторна робота №2. RAG з нуля і контрольований ablation

25 серпня 2026 р.

Мета

Побудувати локальний пошуковий пайплайн над відкритими документами і експериментально показати, що дає кожна зміна конфігурації, вимірюючи це на одному наборі запитів із ручною розміткою.

Робота спирається на лекції 6–8. Аудиторний час — 4 години, самостійна робота — 8 годин.

Теоретичні відомості

Retrieval-augmented generation з'єднує пошук по зовнішньому сховищу з генерацією: модель отримує знайдені фрагменти в контексті і будує відповідь із них, а не з ваг. Оцінювати таку систему як одне ціле марно, бо ламаються дві різні речі: пошук може не знайти потрібний фрагмент, або знайти, а модель його проігнорує. Тому retriever і generator міряються окремо.

- **Одиницею пошуку є фрагмент, а не документ.** Разом із текстом фрагмента зберігаються метадані: джерело, розділ, дата, права доступу. Без них неможливі ні фільтрація, ні цитування, ні видалення застарілого.
- **Розріджений пошук зіставляє слова.** Оцінка BM25 зростає з частотою терміна в документі та його рідкісністю в колекції і нормується на довжину:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{idf}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 (1 - b + b \frac{|d|}{\text{avgl}})}$$

де $f(t, d)$ — частота терміна, $|d|$ — довжина документа, avgl — середня довжина, k_1 керує насиченням частоти, b — силою нормування на довжину. Такий пошук сильний на точних термінах, кодах і назвах, і слабкий на перефразуванні.

- **Щільний пошук зіставляє змісти.** Запит і фрагмент переводяться в вектори однією моделлю, близькість міряється косинусом кута. Порівнювати косинуси векторів різних моделей не можна — це різні простори. Повний перебір по мільйону векторів не масштабується, тому будується наближений індекс. У HNSW параметр M задає кількість ребер вузла, `efConstruction` — ширину пошуку при побудові, `efSearch` — при запиті; зростання `efSearch` підвищує повноту і затримку одночасно.
- **Два підходи помиляються по-різному, тому їх об'єднують.** Reciprocal Rank Fusion не потребує узгодження шкал, бо працює з рангами:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60$$

де R — множина списків видачі, $\text{rank}_r(d)$ — позиція документа в списку r . Константа k пригнічує вплив перших позицій і робить злиття стійким.

- **Реранкер ставлять другим етапом.** Бієнкодер кодує запит і документ окремо, тому вектори документів обчислюються заздалегідь; кросенкодер подає пару на вхід разом і тому точніший, але вимагає прогону моделі на кожну пару і застосовний лише до кількох десятків кандидатів.
- **Порівнюють не два окремі інтервали, а інтервал різниці.** Дві конфігурації, виміряні на тому самому наборі, дають парні спостереження: для кожного запиту рахується Recall@5 обох конфігурацій і їхня різниця, а інтервал будується вже для середньої різниці — наприклад, парним bootstrap на 10 000 ресемплів. Дисперсія парної різниці менша, ніж у кожної оцінки окремо, тому такий тест бачить менші ефекти. Висновок «інтервали перекриваються, отже різниці немає» некоректний: перекриття маргінальних інтервалів не є критерієм. Окремо врахуйте кластери: десять запитів до одного документа — це одне спостереження з десятьма гранями, тому ресемпл робиться за документами.

Корисні посилання для ознайомлення:

- Lewis P. та ін. [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#), NeurIPS 2020;
- Gao Y. та ін. [Retrieval-Augmented Generation for LLMs: A Survey](#);
- Anthropic, [Introducing Contextual Retrieval](#);
- FAISS: [Guidelines to choose an index](#).

Порядок виконання

Позначка *ауд.* означає, що крок виконується на занятті, *сам.* — самостійно. Збирання корпусу і ручна розмітка 60 запитів свідомо винесені на домашній етап: на парі на них часу немає.

1. **Зібрати корпус** (40 хв, сам.): 30–100 відкритих документів однієї предметної області з явно зазначеною ліцензією. Джерела і ліцензії звести в таблицю.

Перевірка: документ без зрозумілої ліцензії до корпусу не потрапляє.

2. **Реалізувати ingestion** (60 хв, ауд.): парсинг, очищення, чанкінг із перекриттям, збереження метаданих (джерело, сторінка або розділ, дата). Обґрунтувати обраний розмір чанка, порахувати кількість чанків і розмір індексу в мегабайтах.

Перевірка, яка коштує хвилину і рятує день: узяти точну фразу з паперового документа і пошукати її серед чанків. Не знайшли — далі йти немає сенсу.

3. **Скласти розмічений набір** (60 хв, сам.): не менш ніж 60 запитів природною мовою, для кожного вручну позначені правильні чанки. Набір фіксується до початку експериментів і більше не змінюється.

Перевірка: запити взяті з того, як питали б реальні користувачі, а не придумані під наївний індекс.

4. **Побудувати два індекси** (40 хв, ауд.): розріджений (`rank_bm25` або аналог) і щільний (`sentence-transformers` з невеликою багатомовною моделлю, `FAISS` або `sqlite-vec`).

Дотриматися контракту обраної моделі: для сімейства E5 обов’язкові префікси `query:` і `passage:`; для косинуса у `FAISS` — нормалізація векторів разом із `METRIC_INNER_PRODUCT`, інакше індекс рахує евклідову відстань, а не те, що ви вважаєте косинусом.

5. **Виміряти чотири конфігурації** (60 хв, ауд.) на цьому наборі:

- лише BM25;
- лише щільний пошук;
- гібрид через RRF;
- гібрид із кросенкодерним перанкером `top-20` → `top-5`.

Для кожної навести `Recall@5`, `MRR` і затримку `p50`. *Змінювати рівно одну річ за раз* — інакше приріст нема кому приписати.

6. **Порахувати довірчі інтервали** (30 хв, ауд.). Для кожної пари конфігурацій — 95 % інтервал парної різниці `per-query Recall@5` (парний `bootstrap`, ресемпл за документами). Явно сказати, які пари розрізняються, а в яких інтервал накриває нуль.

Перевірка: назвати пари, різниця яких накриває нуль. Це головний результат роботи, а не невдача.

7. **Провести власний ablation** (40 хв, сам.): одна зміна за вибором — розмір чанка, перекриття, кількість кандидатів або `efSearch`. Змінити рівно один параметр, набір лишити тим самим.

8. **Додати генерацію з цитуванням** (40 хв, ауд.): кожне твердження супроводжується вказівником на чанк, а за відсутності релевантного контексту система відмовляється відповідати. Перевірити на 10 запитах, серед яких два свідомо поза корпусом.

Перевірка: при порожньому пошуку система відмовляється, а не видає.

9. **Здати роботу** (40 хв, сам.): код і README, набір запитів із розміткою, таблицю чотирьох конфігурацій із інтервалами парних різниць, результат додаткового ablation, 10 прикладів відповідей із цитатами і висновок до 200 слів про те, яка зміна дала найбільший приріст на одиницю затримки.

Порядок кроків 3 і 4 не переставляти: набір, складений після того, як побачили видачу індексу, міряє індекс, а не задачу.

Контрольні запитання

1. Чому оцінювання retriever і generator розділяють, і що показує кожна з двох оцінок?
2. Що станеться з BM25, якщо параметр b дорівнює нулю?
3. Чому косинусна близькість між векторами двох різних моделей не має сенсу?
4. Що саме змінює `efSearch` у HNSW і чим за це платять?
5. Навіщо в RRF константа 60 і що буде, якщо взяти 1?
6. Чому кросенкодер не застосовують до всієї бази?
7. Recall@5 зріс, а якість відповідей — ні. Які причини можливі?
8. Як відрізнити випадок «чанк не знайдено» від випадку «чанк знайдено, але зігноровано»?
9. Чому набір запитів фіксується до експериментів, а не складається за їх результатами?
10. Коли пошук взагалі не потрібен і документи можна подати в контекст цілком?

Література

1. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. *FnTIR*, 2009.
2. Malkov Yu., Yashunin D. [Efficient and Robust Approximate Nearest Neighbor Search Using HNSW Graphs](#). *TPAMI*, 2018.
3. Cormack G., Clarke C., Buettcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR*, 2009.
4. Thakur N. та ін. [BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of IR Models](#). *NeurIPS*, 2021.
5. Es S. та ін. [RAGAS: Automated Evaluation of Retrieval Augmented Generation](#), 2023.
6. Jurafsky D., Martin J. H. [Speech and Language Processing](#), 3rd ed. draft, 2026. Розділи 5, 11.